

Unit – 1 : Session – 12 : SLO – 2 | Serialization & Distributed Processing**Activity (Solutions)**

- 1) `class Student implements Serializable { String name; int marks; Student(String n, int m){ name=n; marks=m; } }` Serialize: `ObjectOutputStream o = new ObjectOutputStream(new FileOutputStream("s.dat")); o.writeObject(new Student("Sam",90)); o.close();` Deserialize: `ObjectInputStream in = new ObjectInputStream(new FileInputStream("s.dat")); Student s = (Student) in.readObject();`
- 2) `serialVersionUID`: a version number for the `Serializable` class; during deserialization the JVM checks that the stored `serialVersionUID` matches the current class's — mismatch throws `InvalidClassException`. Declaring `private static final long serialVersionUID = 1L;` keeps control over compatibility when the class evolves.
- 3) Serialized objects in file I/O: write a list of Students to a file with `ObjectOutputStream` (as above) so objects persist between runs or travel across a network (distributed processing); on restart, `ObjectInputStream` rebuilds the exact objects — enabling save/load and sending objects between JVMs.